

สรุปหลักการ Stack และ Queue

Stack เป็นโครงสร้างข้อมูลเชิงเส้น ที่ทำงานตามหลักการ LIFO (Last in First Out) หรือ “เข้าทีหลัง ออกก่อน” โดยข้อมูลที่ถูกเพิ่มเข้ามาล่าสุดจะถูกนำออกก่อนเสมอโดยมีคำสั่งสำหรับการทำงานดังนี้

คำสั่งหลักของ Stack ได้แก่

Push: เพิ่มข้อมูลเข้าสู่ Stack

Pop: นำข้อมูลออกจาก Stack

Show Stack: แสดงข้อมูลใน Stack

Clear Stack: ล้างข้อมูลทั้งหมดใน Stack

จากโค้ดรวมกันจะมีตัวแปรที่สำคัญคือ Top ใช้ระบุตำแหน่งของข้อมูลบนสุดใน Stack ตัวอย่างเช่น

“Push 10 Push 20 Push 30 Pop” ผลลัพธ์ “30”

ตัวอย่างในชีวิตประจำวัน เช่น การซ้อนจาน การวางหนังสือ การซ้อนถาดในโรงอาหาร

Queue เป็นโครงสร้างข้อมูลเชิงเส้นที่ทำงานตามหลักการ FIFO (First in First Out) หรือ “เข้าก่อน ออกก่อน” ข้อมูลที่ถูกเพิ่มเข้ามาข้อมูลแรกจะถูกนำออกก่อนเสมอโดยมีคำสั่งสำหรับการทำงานดังนี้

คำสั่งหลักของ Queue ได้แก่

Insert: เพิ่มข้อมูลเข้าสู่ Queue

Delete: ลบข้อมูลออกจากตำแหน่ง F เข้าใกล้ R

Show Queue: แสดงข้อมูลใน Queue

Clear Queue: ล้างข้อมูลทั้งหมดใน Queue

จากโค้ดรวมกันจะมีตัวแปรที่สำคัญคือ F (Front) กับ R (Rear) โดย F จะชี้ตำแหน่งข้อมูลตัวแรก ส่วน R จะชี้ตำแหน่งข้อมูลตัวล่าสุด ตัวอย่างโค้ดการทำงานเช่น

“Insert 10 Insert 20 Insert 30 Delete” ผลลัพธ์ “10”

ตัวอย่างในชีวิตประจำวัน เช่น การรอรับบริการทั่วไป ระบบเอกสาร ระบบศูนย์บริการลูกค้า

Overflow คือสถานะที่ไม่สามารถเพิ่มข้อมูลใหม่ได้ เนื่องจากพื้นที่เก็บข้อมูลเต็มแล้ว ตัวอย่างเช่น

Push เข้า Stack ที่โครงสร้างข้อมูลเต็ม

Insert เข้า Queue ที่โครงสร้างข้อมูลเต็ม

Underflow คือสถานะที่ไม่สามารถนำข้อมูลออกได้ เนื่องจากไม่มีข้อมูลอยู่ในโครงสร้างข้อมูล ตัวอย่างเช่น
Pop จาก Stack ที่โครงสร้างข้อมูลว่าง
Delete จาก Queue ที่โครงสร้างข้อมูลว่าง

โครงสร้างโปรแกรม

โครงสร้างการทำงานของโปรแกรมจะมีลักษณะดังนี้

Main()

Stack Menu

Push()

Pop()

Showstack()

Clearstack()

Isstackfull()

Isstackempty()

Queue Menu

Insertqueue()

Deletequeue()

Showqueue()

Clearqueue()

Isqueuefull()

Isqueueempty()

Shared Funtions

Showprogramheader()

Showqoperationresult()

Cleararray()

รายชื่อฟังก์ชันและหน้าที่

ฟังก์ชันของ Stack

push(int value) เพิ่มข้อมูลเข้าสู่ Stack

pop(void) นำข้อมูลบนสุดออกจาก Stack

showStack(void) แสดงข้อมูลทั้งหมดใน Stack

clearStack(void) ล้างข้อมูลภายใน Stack

isStackFull(void) ตรวจสอบว่า Stack เต็มหรือไม่

isStackEmpty(void) ตรวจสอบว่า Stack ว่างหรือไม่

ฟังก์ชันของ Queue

insertQueue(int value) เพิ่มข้อมูลเข้าสู่ Queue

deleteQueue(void) นำข้อมูลออกจาก Queue

showQueue(void) แสดงข้อมูลทั้งหมดใน Queue

clearQueue(void) ล้างข้อมูลภายใน Queue

isQueueFull(void) ตรวจสอบว่า Queue เต็มหรือไม่

isQueueEmpty(void) ตรวจสอบว่า Queue ว่างหรือไม่

ฟังก์ชันที่ Stack และ Queue ใช้ร่วมกัน

showProgramHeader() ใช้แสดงหัวข้อของโปรแกรมหรือสถานะของโครงสร้างของข้อมูลจะมีประโยชน์ในเรื่องของการลดการเขียน printf ซ้ำ และสามารถไปใช้ได้ทั้ง Stack และ Queue

showOperationResult() ใช้สำหรับการแสดงผลการทำงานในส่วนของ push, pop, insert, delete มีโยชน์ในเรื่องรูปแบบการแสดงผลเหมือนกัน และสามารถแก้ไขได้จากจุดเดียว

clearArray() ใช้สำหรับล้างเนื้อหาภายในโครงสร้างข้อมูลอย่าง Array โดยใช้ได้ทั้ง stack[] และ queue[] จะมีประโยชน์ในด้านการช่วยลดการเกิดโค้ดซ้ำ

Source Code

```
#include <stdio.h>
#define MAX_SIZE 5
/* =====
    Global Data
===== */
```

```

int stack[MAX_SIZE];
int top = -1; /*1
int queue[MAX_SIZE];
int front = 0;
int rear = -1;
int queueCount = 0;
/* =====
    Shared Functions
===== */
void showProgramHeader(const char title[]) {
    printf("\n===== %s =====\n",title);
}
void clearArray(int arr[], int size) {
    int i;
    for(i=0;i<size;i++) {
        arr[i] = 0;
    }
}
void showOperationResult (
    const char operation[],
    const char detail[],
    const char before[],
    const char after[],int success) {
    printf("\nOperation : %s\n", operation);
    printf("%s\n", detail);
    printf("Before   : %s\n", before);
    printf("After    : %s\n", after);
    if(success) {
        printf("Result   : SUCCESS\n");
    } else {
        printf("Result   : FAILED\n");
    }
}

```

```

/* =====

Stack Functions
===== */
int isStackFull(void) {
    return (top == MAX_SIZE - 1);
}
int isStackEmpty(void) {
    return (top == -1);
}
void push(int value) {
    char before[100];
    char after[100];
    char detail[100];
    sprintf(before,"Top = %d, Data = %d",
            top, top + 1);
    if(isStackFull()) {
        printf("\nPush failed: Stack Overflow\n");
        return;
    }
    top++;
    stack[top] = value;
    sprintf(after,"Top = %d, Data = %d", top, top + 1);
    sprintf(detail,"Input    : %d", value);
    showOperationResult(
        "PUSH",
        detail,
        before,
        after,
        1
    );
}
int pop(void) {
    int value;

```

```

char before[100];
char after[100];
char detail[100];
sprintf(before,"Top = %d, Data = %d", top, top + 1);
if(isStackEmpty()) {
    printf("\nPop failed: Stack Underflow\n");
    return -9999;
}
value = stack[top];
stack[top] = 0;
top--;
sprintf(after,"Top = %d, Data = %d",top,top + 1);
sprintf(detail,"Output   : %d",value);
showOperationResult("POP",detail,before,after,1);
return value;
}

void clearStack(void) {
    clearArray(stack, MAX_SIZE);
    top = -1;
    printf("\nStack cleared successfully.\n");
}

void showStack(void) {
    int i;
    showProgramHeader("STACK STATUS");
    printf("Capacity : %d\n", MAX_SIZE);
    printf("Data      : %d\n", top + 1);
    printf("Top       : %d\n", top);
    if(isStackEmpty()) {
        printf("Status   : EMPTY\n");
        printf("-----\n");
        printf("Stack is empty.\n");
        printf("-----\n");
        return;
    }
}

```

```

    }
    printf("Status : NORMAL\n");
    printf("-----\n");
    for(i=top;i>=0;i--) {
        if(i==top) {
            printf("TOP -> [%d] index %d\n",stack[i], i);
        } else {
            printf("    [%d] index %d\n",stack[i], i);
        }
    }
    printf("-----\n");
}

/* =====
Queue Functions
===== */
int isQueueFull(void) {
    return (queueCount == MAX_SIZE);
}
int isQueueEmpty(void) {
    return (queueCount == 0);
}
void insertQueue(int value) {
    char before[100];
    char after[100];
    char detail[100];
    sprintf(before,"Front = %d, Rear = %d, Data = %d",front,rear,queueCount);
    if(isQueueFull()) {
        printf("\nInsert failed: Queue Overflow\n");
        return;
    }
    rear++;
    queue[rear] = value;
    queueCount++;

```

```

    printf(after,"Front = %d, Rear = %d, Data = %d",front,rear,queueCount);
    printf(detail,"Input    : %d",value);
    showOperationResult("INSERT QUEUE",detail,before,after,1);
}

int deleteQueue(void) {
    int value;
    char before[100];
    char after[100];
    char detail[100];
    printf(before,"Front = %d, Rear = %d, Data = %d",front,rear,queueCount);
    if(isQueueEmpty()) {
        printf("\nDelete failed: Queue Underflow\n");
        return -9999;
    }
    value = queue[front];
    queue[front] = 0;
    front++;
    queueCount--;
    if(queueCount == 0) {
        front = 0;
        rear = -1;
    }
    printf(after,"Front = %d, Rear = %d, Data = %d",front,rear,queueCount);
    printf(detail,"Output   : %d",value);
    showOperationResult("DELETE QUEUE",detail,before,after,1);
    return value;
}

void clearQueue(void) {
    clearArray(queue, MAX_SIZE);
    front = 0;
    rear = -1;
    queueCount = 0;
    printf("\nQueue cleared successfully.\n");
}

```



```

}

void showQueue(void) {
    int i;
    showProgramHeader("QUEUE STATUS");
    printf("Capacity : %d\n", MAX_SIZE);
    printf("Data      : %d\n", queueCount);
    printf("Front    : %d\n", front);
    printf("Rear     : %d\n", rear);
    if(isQueueEmpty()) {
        printf("Status   : EMPTY\n");
        printf("-----\n");
        printf("Queue is empty.\n");
        printf("-----\n");
        return;
    }
    printf("Status   : NORMAL\n");
    printf("-----\n");
    printf("Index : ");
    for(i=0;i<MAX_SIZE;i++) {
        printf(" %3d ", i);
    }
    printf("\n");
    printf("Data : ");
    for(i=0;i<MAX_SIZE;i++) {
        if(queue[i]==0) {
            printf("[ ] ");
        } else {
            printf("[%d] ", queue[i]);
        }
    }
    printf("\n");
    printf("      ");
    for(i = 0 ; i < MAX_SIZE ; i++) {

```

```

        if(i == front && i == rear) {
            printf("FR  ");
        } else if(i == front) {
            printf("  F  ");
        } else if(i == rear) {
            printf("  R  ");
        } else {
            printf("    ");
        }
    }
    printf("\n");
    printf("Order : ");
    for(i=front;i<=rear;i++) {
        printf("%d", queue[i]);
        if(i<rear) {
            printf(" -> ");
        }
    }
    printf("\n");
    printf("-----\n");
}

/* =====
Menu
===== */
void stackMenu(void) {
    int choice;
    int value;
    do {
        printf("\n===== STACK MENU =====\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Show Stack\n");
        printf("4. Clear Stack\n");
    } while(1);
}

```

```

printf("0. Back to Main Menu\n");
printf("-----\n");
printf("Select menu : ");
scanf("%d",&choice);
switch(choice) {
    case 1:
        printf("Input value : ");
        scanf("%d",&value);
        push(value);
        break;
    case 2:
        pop();
        break;
    case 3:
        showStack();
        break;
    case 4:
        clearStack();
        break;
    case 0:
        break;
    default:
        printf("\nInvalid menu.\n");
}
} while(choice!=0);
}

void queueMenu(void) {
    int choice;
    int value;
    do {
        printf("\n===== QUEUE MENU =====\n");
        printf("1. Insert\n");
        printf("2. Delete\n");

```

```

printf("3. Show Queue\n");
printf("4. Clear Queue\n");
printf("0. Back to Main Menu\n");
printf("-----\n");
printf("Select menu : ");
scanf("%d",&choice);
switch(choice) {
    case 1:
        printf("Input value : ");
        scanf("%d",&value);
        insertQueue(value);
        break;
    case 2:
        deleteQueue();
        break;
    case 3:
        showQueue();
        break;
    case 4:
        clearQueue();
        break;
    case 0:
        break;
    default:
        printf("\nInvalid menu.\n");
}
} while(choice!=0);
}

int main(void) {
    int choice;
    do {
        printf("\n=====");
        printf("    STACK AND QUEUE PROGRAM\n");
    }
}

```

```
printf("=====\n");
printf("1. Stack\n");
printf("2. Queue\n");
printf("0. Exit\n");
printf("-----\n");
printf("Select menu : ");
scanf("%d",&choice);
switch(choice) {
    case 1:
        stackMenu();
        break;
    case 2:
        queueMenu();
        break;
    case 0:
        break;
    default:
        printf("\nInvalid menu.\n");
}
} while(choice!=0);
return 0;
}
```

ผลการทดสอบ

ชุดทดสอบที่ 1 : Stack ทำงานตามปกติ

กำหนด Stack ขนาด 5 ช่อง

```
Push 10
Push 20
Push 30
Pop
Push 40
Show Stack
```

1. ข้อมูลใดถูก Pop ออก

ข้อมูลที่ Pop ออก = 30

2. ข้อมูลที่เหลือใน Stack เรียงจาก Top ลงมาเป็นอย่างไร

ข้อมูลคงเหลือ

40

20

10

3. ค่า Top สุดท้ายเป็นเท่าใด

ค่า Top สุดท้ายเป็น 2

ชุดทดสอบที่ 2 : Stack Overflow

Clear Stack

Push 11

Push 22

Push 33

Push 44

Push 55

Push 66

ให้ตอบว่า

1. คำสั่งใดทำให้เกิด Overflow

```
if(isStackFull()) {
    printf("\nPush failed: Stack Overflow\n");
    return;
}
```

2. ข้อมูล 66 ถูกเก็บลง Stack หรือไม่

66 ไม่ถูกเก็บ

3. จำนวนข้อมูลสุดท้ายใน Stack เท่ากับเท่าใด
ข้อมูลสุดท้ายมีค่าเท่ากับ 5 รายการ

ชุดทดสอบที่ 3 : Stack Underflow

Clear Stack

Pop

ให้ตอบว่า

1. เพราะเหตุใดจึงเกิด Underflow
ภายในโครงสร้างข้อมูลไม่พบข้อมูลปัจจุบันเหลือแล้ว
2. โปรแกรมยังสามารถกลับไปแสดงเมนูได้หรือไม่
โปรแกรมยังสามารถกลับสู่เมนูได้

ชุดทดสอบที่ 4 : Queue ทำงานตามปกติ

Insert 10

Insert 20

Insert 30

Delete

Insert 40

Show Queue

ให้ตอบว่า

1. ข้อมูลใดถูก Delete ออก
ข้อมูล 10 จะถูก Delete ออก
2. ข้อมูลที่เหลืออยู่ใน Queue เรียงตามลำดับอย่างไร
ข้อมูลที่เหลือ 20 -> 30 -> 40
3. ค่า Front และ Rear สุดท้ายเป็นเท่าใด
 $\text{Front} = 1$ $\text{Rear} = 3$

ชุดทดสอบที่ 5 : Queue Overflow

Clear Queue

Insert 11

Insert 22

Insert 33

Insert 44

Insert 55

Insert 66

ให้ตอบว่า

1. คำสั่งใดทำให้เกิด Overflow

```
if(isQueueFull()) {  
    printf("\nInsert failed: Queue Overflow\n");  
    return;  
}
```

2. ข้อมูล 66 ถูกเก็บใน Queue หรือไม่

ข้อมูล 66 จะไม่ถูกจัดเก็บลงในโครงสร้างข้อมูล

3. จำนวนข้อมูลสุดท้ายใน Queue เท่ากับเท่าใด

ข้อมูลสุดท้ายของโครงสร้างข้อมูลจะมีค่าเท่ากับ 5 รายการ

ชุดทดสอบที่ 6 : Queue Underflow

Clear Queue

Delete

ให้ตอบว่า

1. เพราะเหตุใดจึงเกิด Underflow

ภายในโครงสร้างข้อมูลไม่พบข้อมูลปัจจุบันเหลือแล้ว

2. โปรแกรมจัดการกับค่าที่ไม่มีอยู่ใน Queue อย่างไร

โปรแกรมจะเกิดข้อผิดพลาดซึ่งเข้าใจผิดที่รองรับไว้ก็คือ Underflow

ชุดทดสอบที่ 7 : เปรียบเทียบ Stack และ Queue

ให้ใช้ข้อมูลชุดเดียวกันคือ

10, 20, 30, 40

ดำเนินการดังนี้

1. เพิ่มข้อมูลทั้งหมดเข้าสู่ Stack
2. นำข้อมูลออกจาก Stack จนครบ
3. เพิ่มข้อมูลทั้งหมดเข้าสู่ Queue
4. นำข้อมูลออกจาก Queue จนครบ
5. บันทึกลำดับข้อมูลที่ออกจากแต่ละโครงสร้าง

โครงสร้าง	ลำดับข้อมูลเข้า	ลำดับข้อมูลออก	หลักการ
Stack	10,20,30,40	40,30,20,10	LIFO
Queue	10,20,30,40	10,20,30,40	FIFO

คำตอบคำถามท้ายใบงาน

1. Stack และ Queue แตกต่างกันอย่างไรร

Stack และ Queue เป็นโครงสร้างข้อมูลเชิงเส้นเหมือนกัน แต่มีวิธีนำข้อมูลออกแตกต่างกัน

Stack ใช้หลักการ LIFO (Last In First Out) ข้อมูลที่เข้าเป็นตัวสุดท้ายจะออกก่อน

Queue ใช้หลักการ FIFO (First In First Out) ข้อมูลที่เข้าก่อนจะออกก่อน

2. เพราะเหตุใด Stack จึงใช้ตัวชี้ตำแหน่งเพียงตัวเดียว

เพราะ Stack เพิ่มและนำข้อมูลออกที่ตำแหน่งเดียวกันเสมอ คือด้านบนสุดของ Stack (Top)

ดังนั้นจึงใช้เพียงตัวแปร top หรือ SP ก็เพียงพอสำหรับควบคุมการทำงานทั้งหมด

3. เพราะเหตุใด Queue จึงต้องมีทั้ง Front และ Rear

เพราะ Queue เพิ่มข้อมูลและนำข้อมูลออกคนละด้าน

Front ใช้ระบุตำแหน่งข้อมูลตัวแรก

Rear ใช้ระบุตำแหน่งข้อมูลตัวล่าสุด

จึงต้องใช้ตัวชี้ 2 ตัวในการจัดการข้อมูล

4. Overflow และ Underflow แตกต่างกันอย่างไรร

Overflow เกิดเมื่อพยายามเพิ่มข้อมูลเข้าไปในโครงสร้างข้อมูลที่เต็มแล้วดังนี้

push ข้อมูลเข้า stack ที่เต็ม

insert ข้อมูลเข้า queue ที่เต็ม

Underflow เกิดเมื่อพยายามนำข้อมูลออกจากโครงสร้างข้อมูลที่ว่างดังนี้

pop จาก stack ว่าง

Delete จาก queue ว่าง

5. ฟังก์ชัน isStackFull() มีหน้าที่อะไร

ใช้ตรวจสอบว่า Stack เต็มหรือไม่ ก่อนทำการ Push จากโค้ดคือ if(isStackFull())

หากคืนค่าเป็นจริง จะไม่อนุญาตให้เพิ่มข้อมูล และแสดงข้อความ Overflow

6. ฟังก์ชัน isEmpty() มีหน้าที่อะไร

ใช้ตรวจสอบว่า Queue ว่างหรือไม่ ก่อนทำการ Delete จากโค้ดคือ if(isQueueEmpty())

หาก Queue ว่าง จะไม่อนุญาตให้นำข้อมูลออกและแสดงข้อความ Underflow

7. เพราะเหตุใดจึงไม่ควรเขียนคำสั่งทั้งหมดไว้ใน main()

เพราะจะทำให้โปรแกรม อ่านยาก แก้ไขยาก มีโค้ดจำนวนมากในที่เดียว นำกลับมาใช้ซ้ำไม่ได้

การแยกเป็นฟังก์ชันจะช่วยให้โปรแกรมเป็นระเบียบและบำรุงรักษาได้ง่ายกว่า

8. ส่วนใดของโปรแกรม Stack และ Queue สามารถใช้ร่วมกันได้

ตัวอย่างส่วนที่ใช้ร่วมกันได้ แสดงหัวข้อโปรแกรม แสดงผลการทำงาน รับข้อมูลจากผู้ใช้
แสดงข้อความผิดพลาด แสดงเส้นคั่น ล้างข้อมูลใน Array จากโค้ดคือ showProgramHeader()
และ showOperationResult()

9. การแสดงค่า Index ของ Array ช่วยให้เข้าใจโปรแกรมอย่างไร

ช่วยให้ทราบว่า ข้อมูลแต่ละตัวถูกเก็บไว้ที่ตำแหน่งใด Front, Rear และ Top ชี้ไปตำแหน่งใด
การเปลี่ยนแปลงตำแหน่งหลัง Push, Pop, Insert และ Delete
จึงช่วยให้เข้าใจการทำงานภายในของโปรแกรมได้ง่ายขึ้น

10. ถ้าต้องเพิ่มขนาด Stack และ Queue จาก 5 เป็น 10 ต้องแก้ไขส่วนใด
แก้ไขค่าคงที่เพียงจุดเดียวจากโค้ดคือ #define MAX_SIZE 5 เปลี่ยนเป็น #define MAX_SIZE
10 Array และฟังก์ชันทั้งหมดจะใช้ขนาดใหม่โดยอัตโนมัติ

11. การ Push และ Pop มี Time Complexity เท่าใด

Push $O(1)$ Pop $O(1)$ เนื่องจากเข้าถึงตำแหน่ง Top ได้โดยตรง

12. การ Insert และ Delete ของ Queue มี Time Complexity เท่าใด

Insert $O(1)$ Delete $O(1)$ เนื่องจากทำงานที่ตำแหน่ง Front และ Rear โดยตรง

13. ยกตัวอย่างการใช้งาน Stack ในชีวิตจริงหรือระบบคอมพิวเตอร์ 2 ตัวอย่าง

ตัวอย่างที่ 1 ระบบ Undo ของโปรแกรม Microsoft Word

“คำสั่งล่าสุดถูกยกเลิกก่อน”

ตัวอย่างที่ 2 การจัดการ Function Call ในโปรแกรม

“ฟังก์ชันที่ถูกเรียกล่าสุดจะถูกคืนค่ากลับก่อน”

14. ยกตัวอย่างการใช้งาน Queue ในชีวิตจริงหรือระบบคอมพิวเตอร์ 2 ตัวอย่าง

ตัวอย่างที่ 1 การต่อคิวซื้อสินค้า

“ใครมาก่อน ได้รับบริการก่อน”

ตัวอย่างที่ 2 ระบบ Print Queue ของเครื่องพิมพ์

“เอกสารที่ส่งพิมพ์ก่อน จะถูกพิมพ์ก่อน”

ปัญหาที่พบและวิธีแก้ไข

ปัญหาที่ 1 การรวม Stack และ Queue ไว้ในโปรแกรมเดียวทำให้โค้ดมีความซับซ้อนเพิ่มขึ้น

วิธีแก้ แบ่งโค้ดออกเป็นฟังก์ชันย่อยตามหน้าที่

ปัญหาที่ 2 มีโค้ดแสดงผลซ้ำกันหลายส่วน

วิธีแก้ สร้างฟังก์ชันกลาง showProgramHeader(), showOperationResult() และ clearArray()

เพื่อลดความซ้ำซ้อน

ปัญหาที่ 3 การจัดรูปแบบการแสดงผลให้เห็น Front, Rear และ Top อย่างชัดเจน
วิธีแก้ ออกแบบฟังก์ชันแสดงผลใหม่ให้แสดงตำแหน่งของข้อมูลพร้อมค่า Index

สรุปผลการปฏิบัติ

จากการปฏิบัติการครั้งนี้ทำให้เข้าใจหลักการทำงานของ Stack และ Queue อย่างชัดเจน โดย Stack ใช้หลักการ LIFO ส่วน Queue ใช้หลักการ FIFO นอกจากนี้ยังได้ศึกษาแนวคิดของ Overflow และ Underflow และสามารถนำความรู้ดังกล่าวมาพัฒนาโปรแกรมภาษา C ที่รวม Stack และ Queue ไว้ในโปรแกรมเดียวได้สำเร็จ

ผู้จัดทำได้ฝึกการแบ่งโปรแกรมออกเป็นฟังก์ชันย่อย การสร้างฟังก์ชันที่สามารถใช้ร่วมกันได้ และการออกแบบการแสดงผลที่ช่วยให้เข้าใจการเปลี่ยนแปลงของข้อมูลภายในโครงสร้างข้อมูลได้ง่ายขึ้น ส่งผลให้ความเข้าใจทั้งในด้านทฤษฎีและการประยุกต์ใช้งานจริงของโครงสร้างข้อมูลมากยิ่งขึ้น